

PROJECT REQUIREMENTS DOCUMENT

Phase 1: Whisper + Advanced Speaker Diarization Pipeline

On-Premises GPU Implementation

1 Project Overview

This document outlines the requirements and deliverables for **Phase 1** of the Whisper + Advanced Speaker Diarization Pipeline project. Phase 1 focuses exclusively on developing the core transcription and speaker diarization engines as standalone Python modules, without web interface implementation.

2 Scope of Phase 1

2.1 Included in Phase 1

- Whisper-based transcription engine with GPU acceleration
- Advanced multi-model speaker diarization system
- Integration pipeline combining transcription and diarization
- Command-line interface for testing and validation
- Output generation in multiple formats (JSON, TXT, CSV)

2.2 Excluded from Phase 1

- Web interface development
- Web integration
- User interface components
- File upload mechanisms
- Progress indicators or web-based status displays

3 Technical Requirements

3.1 Requirement 1: Transcription Engine

Core Technology: OpenAI Whisper or WhisperX with CUDA acceleration

Functional Requirements:

- Process audio files in MP3 and WAV formats
- Automatic chunking for long-form audio processing
- Seamless stitching of transcribed segments
- Language auto-detection and manual language selection support
- Word-level timestamping in output

- GPU optimization for RTX 4090 hardware

Technical Specifications:

- CUDA-enabled PyTorch implementation
- Memory-efficient processing for large files
- Configurable model sizes (tiny, base, small, medium, large)
- Support for German language optimization

3.2 Requirement 2: Speaker Diarization Module

Multi-Model Approach: Implementation beyond PyAnnote alone

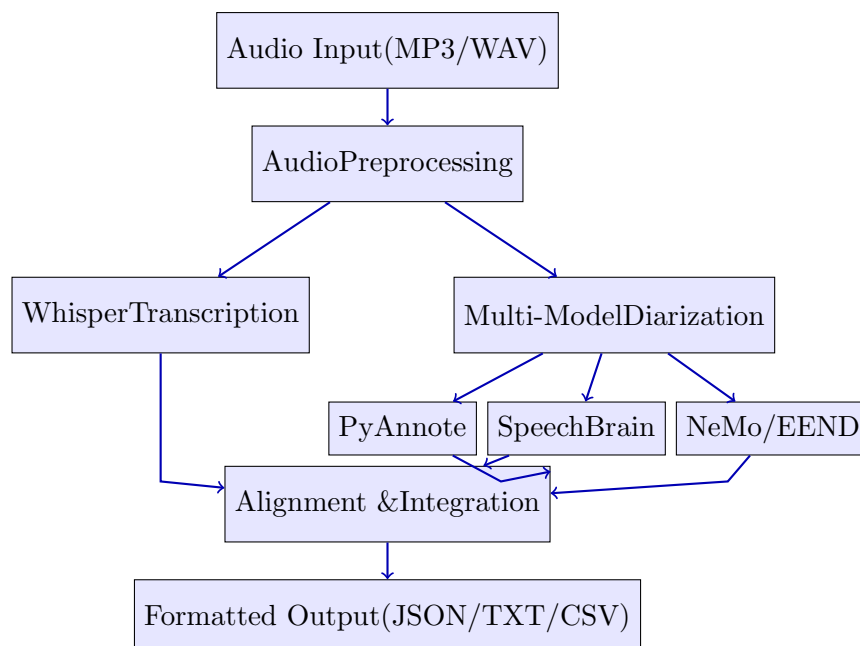
Required Diarization Technologies:

- PyAnnote (baseline implementation)
- SpeechBrain speaker diarization
- NeMo speaker diarization toolkit
- EEND-based solutions (DiariZen or ESPnet)

Functional Requirements:

- Robust multi-speaker detection and separation
- Consistent speaker labeling across audio duration
- Speaker embedding generation and clustering
- Alignment synchronization with Whisper timestamps
- Ensemble approach for improved accuracy

4 System Architecture



5 Integration Pipeline

5.1 Processing Workflow

1. Audio file validation and preprocessing
2. Parallel execution of Whisper transcription and multi-model diarization
3. Timestamp alignment between transcript and speaker segments
4. Speaker label consistency across diarization models
5. Output generation in requested formats

5.2 Alignment Strategy

- Word-level timestamp matching
- Speaker boundary optimization
- Confidence scoring for speaker assignments
- Conflict resolution between multiple diarization outputs

6 Deliverables

6.1 Core Components

- `whisper_engine.py` - Transcription module with GPU optimization
- `diarization_ensemble.py` - Multi-model speaker diarization system
- `pipeline_integration.py` - Main integration and processing pipeline
- `output_formatter.py` - Multi-format output generation
- `cli_interface.py` - Command-line interface for testing

6.2 Output Formats

- **JSON:** Structured data with nested speaker segments and timestamps
- **TEXT:** Plain text with speaker labels and time markers
- **CSV:** Tabular format with columns for time, speaker, and text

7 Technical Constraints

7.1 Hardware Requirements

- Target deployment on RTX 4090 GPU
- CUDA 11.8+ compatibility

7.2 Security and Privacy

- Complete offline operation (no external API calls)
- Local model storage and inference
- GDPR compliance for audio data processing
- Secure temporary file handling

8 Testing and Validation

8.1 Test Cases

- Single speaker German audio (5-60 minutes)
- Multi-speaker meetings (2-6 speakers, 30-120 minutes)
- Various audio quality levels (16kHz to 48kHz)

8.2 Success Criteria

- Successful processing of provided test audio files
- Accurate speaker separation with consistent labeling
- Word-level timestamp accuracy within 100ms
- Stable GPU memory usage without overflow

9 Project Assumptions

- Client provides RTX 4090 server with appropriate CUDA drivers
- Test audio files will be provided for validation
- Python 3.8+ environment with pip package management

10 Next Steps

1. Client confirmation of requirements understanding
2. Development environment setup and model downloads
3. Implementation of core modules
4. Integration testing with client-provided audio samples
5. Performance optimization and documentation completion

This document serves as the foundation for Phase 1 development.